

Guided Practice

Integrating Agents with LangGraph Using Google A2A Protocol

Overview

In this guided practice, you will orchestrate multiple AI agents using a modular, plug-and-play architecture powered by the Agent-to-Agent (A2A) protocol with LangGraph. You will work with agents built using LangChain, CrewAI, and the OpenAI API, all connected into a single, state-aware workflow.

This exercise demonstrates how user input is parsed, a research task is triggered through web tools, and a final response is synthesized. By the end of this session, you will understand how cross-framework agents collaborate at scale while maintaining shared context, enabling enterprise-ready GenAI workflows.

Instructions

1. Define the agent interaction logic using LangChain, CrewAI, and the OpenAI API
2. Orchestrate the workflow using LangGraph to connect agents
3. Enable modular web research with CrewAI tools
4. Generate final response output through the OpenAI API

Tasks

Task 1: Creating the LangChain agent

Implement a LangChain ReAct agent that extracts the core task from user input. This ensures the system understands intent before passing it downstream.

Task 2: Building the CrewAI research agent

Develop a CrewAI agent that uses a custom SearchTool (backed by Tavily search) to perform web research and return a structured summary of findings.

Task 3: Implementing the custom OpenAI API agent

Define a function that calls OpenAI's chat completion API (GPT-5) with the task and research results. This synthesizes a coherent, human-like response.

Task 4: Orchestrating agents with LangGraph

Construct a stateful graph that links LangChain, CrewAI, and OpenAI nodes. The graph manages input, task extraction, research results, and final output.

Steps to be followed:

1. Set up the environment
2. Create the LangChain agent
3. Build the CrewAI agent
4. Define the custom OpenAI agent
5. Integrate the agents with LangGraph
6. Execute the pipeline

Answer Key

Step 1: Set up the environment

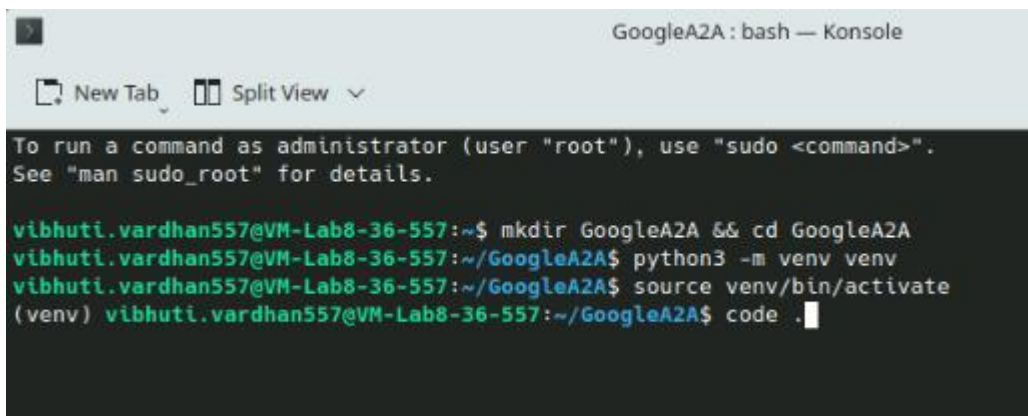
1.1 Open the terminal in your system and run the following commands:

```
mkdir GoogleA2A && cd GoogleA2A
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

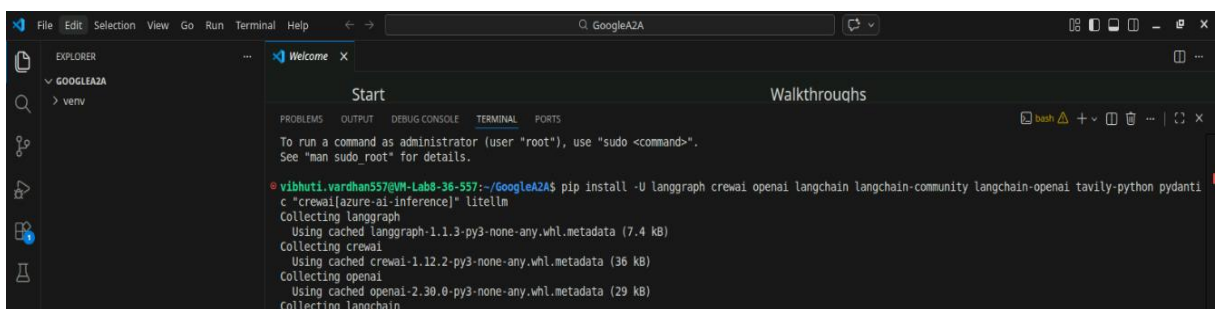
```
code .
```



```
GoogleA2A: bash — Konsole
New Tab Split View
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
vibhuti.vardhan557@VM-Lab8-36-557:~$ mkdir GoogleA2A && cd GoogleA2A
vibhuti.vardhan557@VM-Lab8-36-557:~/GoogleA2A$ python3 -m venv venv
vibhuti.vardhan557@VM-Lab8-36-557:~/GoogleA2A$ source venv/bin/activate
(venv) vibhuti.vardhan557@VM-Lab8-36-557:~/GoogleA2A$ code .
```

1.2 Inside VSCode, open a new terminal and then install and configure the required libraries using the following command:

```
pip install -U langgraph crewai openai langchain langchain-community langchain-openai tavily-python pydantic "crewai[azure-ai-inference]" litellm
```



```
File Edit Selection View Go Run Terminal Help
GoogleA2A
EXPLORER
GOOGLEA2A
venv
Welcome
Start Walkthroughs
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
vibhuti.vardhan557@VM-Lab8-36-557:~/GoogleA2A$ pip install -U langgraph crewai openai langchain langchain-community langchain-openai tavily-python pydantic c "crewai[azure-ai-inference]" litellm
Collecting langgraph
Using cached langgraph-1.1.3-py3-none-any.whl.metadata (7.4 kB)
Collecting crewai
Using cached crewai-1.12.2-py3-none-any.whl.metadata (36 kB)
Collecting openai
Using cached openai-2.30.0-py3-none-any.whl.metadata (29 kB)
Collecting langchain
```

Step 2: Create the LangChain agent

- 2.1 Create a new folder named **agents** and inside it create a new file named **langchain_agent.py** and use the following code to define the LangChain agent:

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import AzureChatOpenAI
import os

@tool
def extract_task_from_input(input_text: str) -> str:
    """Extract the core task from user input."""
    return f"Extracted task from input: {input_text.strip()}"

def create_langchain_agent():
    llm = AzureChatOpenAI(
        azure_deployment="gpt-5-mini",
        api_version="2024-12-01-preview",
        azure_endpoint="https://openai-api-management-gw.azure-api.net/",
        api_key="2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQ
J99BDAC77bzfXJ3w3AAABACOGi3sC",
    )

    tools = [extract_task_from_input]

    agent = create_agent(
        model=llm,
        tools=tools,
        system_prompt="You are a helpful assistant."
    )
    return agent
```

```
langchain_agent.py X
agents > langchain_agent.py
1 from langchain.agents import create_agent
2 from langchain.tools import tool
3 from langchain_openai import AzureChatOpenAI
4 import os
5
6
7 @tool
8 def extract_task_from_input(input_text: str) -> str:
9     """Extract the core task from user input."""
10    return f"Extracted task from input: {input_text.strip()}"
11
12
13 def create_langchain_agent():
14     llm = AzureChatOpenAI(
15         azure_deployment="gpt-5-mini",
16         api_version="2024-12-01-preview",
17         azure_endpoint="https://openai-api-management-gw.azure-api.net/",
18         api_key="2ABecnfzxhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCpsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3sC",
19     )
20
21     tools = [extract_task_from_input]
22
23     agent = create_agent(
24         model=llm,
25         tools=tools,
26         system_prompt="You are a helpful assistant."
27     )
28     return agent
```

Step 3: Build the CrewAI agent

- 3.1 Inside the agents folder, create a new file named **crewai_agent.py** and use the following code to create the CrewAI agent and assign a task:

```
from crewai import Crew, Agent, Task, LLM
from langchain_community.tools.tavily_search import TavilySearchResults
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
import os

AZURE_OPENAI_API_KEY =
"2ABecnfzxhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCpsj0JLKsZPfJQQJ99BDAC77b
zfXJ3w3AAABACOGi3sC"
AZURE_OPENAI_ENDPOINT = "https://openai-api-management-gw.azure-api.net/"
AZURE_OPENAI_API_VERSION = "2024-12-01-preview"
AZURE_OPENAI_DEPLOYMENT = "gpt-5-mini"

TAVILY_API_KEY = "tvly-dev-1uT9rc-
X1i2A54uzUo18Hhz6BL1UjY8KMsU5aSCBbu3VRbNkr"

class ToolInput(BaseModel):
    question: str = Field(description="search query")
```

```

class SearchTool(BaseTool):
    name: str = "Search tool"
    description: str = "use this tool to find current information required for query"
    args_schema: type = ToolInput

    def _run(self, question: str):
        """use this tool to find current information required for query"""
        web_search = TavilySearchResults(
            tavily_api_key=TAVILY_API_KEY
        )
        result = web_search.invoke({"query": question})
        return str(result)

def create_crewai_agent(task_description: str):
    llm = LLM(
        model="azure/gpt-5-mini",
        api_key=AZURE_OPENAI_API_KEY,
        base_url=AZURE_OPENAI_ENDPOINT,
        api_version=AZURE_OPENAI_API_VERSION,
        is_litellm=True,
        max_tokens=3000
    )

    researcher = Agent(
        role="Researcher",
        goal="Search the web and gather information",
        backstory="Expert in web research using advanced techniques",
        tools=[SearchTool()],
        llm=llm,
        verbose=True,
        allow_delegation=False
    )

    task = Task(
        description=f"Research this briefly and return 5 bullet points only: {task_description}",
        expected_output="5 short bullet points with the most relevant findings",
        agent=researcher
    )

```

```

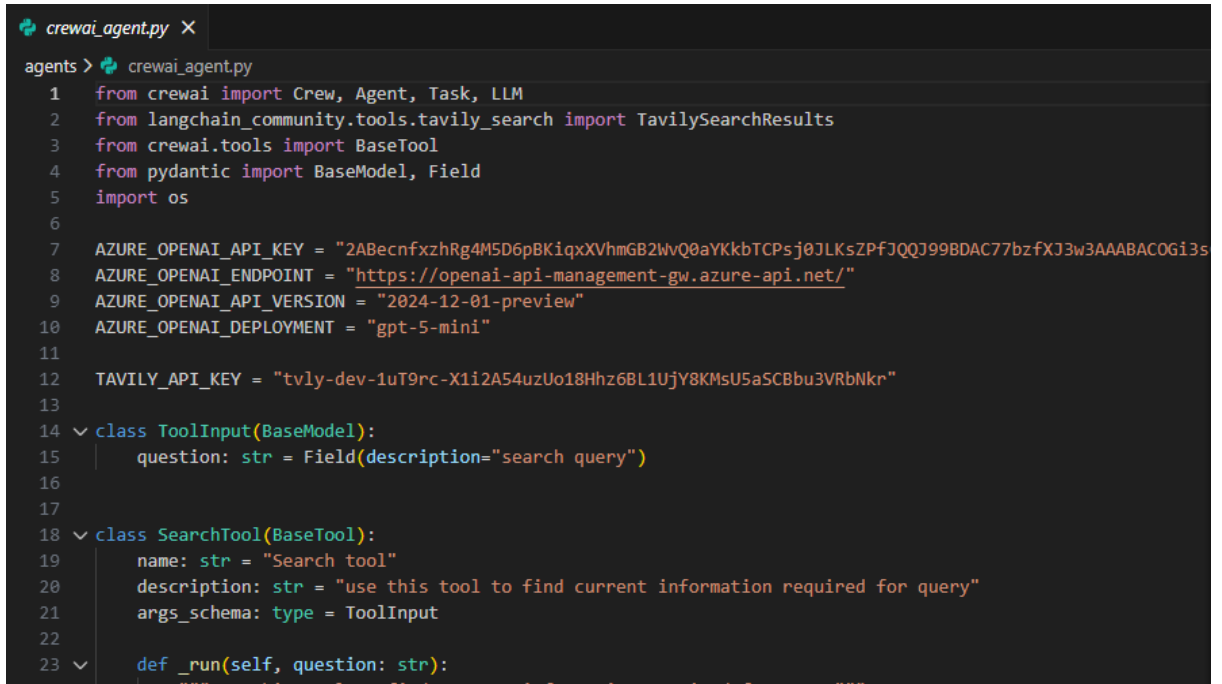
crew = Crew(
    agents=[researcher],
    tasks=[task],
    verbose=True,
)

```

```

return crew

```



```

crewai_agent.py X
agents > crewai_agent.py
1 from crewai import Crew, Agent, Task, LLM
2 from langchain_community.tools.tavily_search import TavilySearchResults
3 from crewai.tools import BaseTool
4 from pydantic import BaseModel, Field
5 import os
6
7 AZURE_OPENAI_API_KEY = "2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3s"
8 AZURE_OPENAI_ENDPOINT = "https://openai-api-management-gw.azure-api.net/"
9 AZURE_OPENAI_API_VERSION = "2024-12-01-preview"
10 AZURE_OPENAI_DEPLOYMENT = "gpt-5-mini"
11
12 TAVILY_API_KEY = "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"
13
14 class ToolInput(BaseModel):
15     question: str = Field(description="search query")
16
17
18 class SearchTool(BaseTool):
19     name: str = "Search tool"
20     description: str = "use this tool to find current information required for query"
21     args_schema: type = ToolInput
22
23     def _run(self, question: str):

```

Step 4: Define the custom OpenAI agent

- 1.1 Inside the agents folder, create a new file named **custom_openai_agent.py** and use the following code to define the custom agent:

```

from crewai import LLM

```

```

llm = LLM(
    model="azure/gpt-5-mini",
    api_key="2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3sC",
    base_url="https://openai-api-management-gw.azure-api.net/",
    api_version="2024-12-01-preview",
    is_litellm=True,

```

)

```
def generate_response(prompt: str) -> str:
    response = llm.call(prompt)
    return str(response)
```



```
custom_openai_agent.py X
agents > custom_openai_agent.py
1  from crewai import LLM
2
3  llm = LLM(
4      model="azure/gpt-5-mini",
5      api_key="2ABecnfzxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3sC",
6      base_url="https://openai-api-management-gw.azure-api.net/",
7      api_version="2024-12-01-preview",
8      is_litellm=True,
9  )
10
11 def generate_response(prompt: str) -> str:
12     response = llm.call(prompt)
13     return str(response)
```

Step 5: Integrate the agent with LangGraph

1.1 Outside the agents folder, create a new file named **main.py** and use the following code to integrate the agents:

```
from typing import TypedDict
```

```
from langgraph.graph import StateGraph, END
```

```
from agents.langchain_agent import create_langchain_agent
```

```
from agents.crewai_agent import create_crewai_agent
```

```
from agents.custom_openai_agent import generate_response
```

```
### Define State
```

```
class A2AState(TypedDict, total=False):
```

```
    input: str
```

```
    task: str
```

```
    web_results: str
```

```
    final_response: str
```

Define Nodes

```
def langchain_node(state):  
    print("state is: ", state)  
    return {"task": state["input"]}
```

```
def crewai_node(state):  
    print(" state task is: ", state["task"])  
    crew = create_crewai_agent(task_description=state["task"])  
    result = crew.kickoff()  
    return {"web_results": str(result)}
```

```
def openai_response_node(state):  
    prompt = f"Based on the web results: {state['web_results']}, respond to the task:  
{state['task']}"  
    response = generate_response(prompt)  
    return {"final_response": response}
```

Build Graph

```
builder = StateGraph(A2AState)  
builder.add_node("LangChainAgent", langchain_node)  
builder.add_node("CrewAI Agent", crewai_node)  
builder.add_node("OpenAI Agent", openai_response_node)
```

Edges

```
builder.set_entry_point("LangChainAgent")  
builder.add_edge("LangChainAgent", "CrewAI Agent")  
builder.add_edge("CrewAI Agent", "OpenAI Agent")  
builder.add_edge("OpenAI Agent", END)
```

```
graph = builder.compile()
```

Execute

```
if __name__ == "__main__":  
    input_text = "Can you summarize the latest trends in GenAI for enterprise?"  
    output = graph.invoke({"input": input_text})  
    print("Final Output:\n", output["final_response"])
```

```
main.py X
main.py
1 from typing import TypedDict
2
3 from langgraph.graph import StateGraph, END
4 from agents.langchain_agent import create_langchain_agent
5 from agents.crewai_agent import create_crewai_agent
6 from agents.custom_openai_agent import generate_response
7
8
9 ### Define State
10 class A2AState(TypedDict, total=False):
11     input: str
12     task: str
13     web_results: str
14     final_response: str
15
16
17 ### Define Nodes
18 def langchain_node(state):
19     print("state is: ", state)
20     return {"task": state["input"]}
21
22
23 def crewai_node(state):
24     print(" state task is: ", state["task"])
25     crew = create_crewai_agent(task_description=state["task"])
26     result = crew.kickoff()
27     return {"web_results": str(result)}
28
29
30 def openai_response_node(state):
31     prompt = f"Based on the web results: {state['web_results']}, respond to the task: {state['task']}"
32     response = generate_response(prompt)
```

Step 5: Execute the pipeline

1.1 Open the terminal and run the following command to execute the code:

python main.py

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

● vibhuti.vardhan557@VM-Lab8-36-557:~/GoogleA2A$ source /home/vibhuti.vardhan557/GoogleA2A/venv/bin/activate
○ (venv) vibhuti.vardhan557@VM-Lab8-36-557:~/GoogleA2A$ python main.py
```

You will see the following output printed in the terminal.

```
C:\Users\SLP13696\Desktop\tested\GPA2A demo\agents\crewai_agent.py:23: LangChainDeprecationWarning: The class `TavilySearchResults` was deprecated in LangChain 0.3.25 and will be removed in 1.0. An updated version of the class exists in the :class:`~langchain-tavily` package and should be used instead. To use it run `pip install -U :class:`~langchain-tavily` and import as `from :class:`~langchain-tavily import TavilySearch`.
  web_search = TavilySearchResults()
Crew: crew
Task: e29f335a-5d0f-4517-b9d7-381735cb89c3
Status: Executing Task...
└─ Used Search tool (1)

Agent Tool Execution

Agent: Researcher

Thought: I need to gather the latest trends in Generative AI (GenAI) for enterprises, focusing on advancements in machine learning algorithms, adoption of AI solutions, and data privacy and security.

Using Tool: Search tool

Tool Input

{"question": "\latest trends in Generative AI for enterprises 2023 machine learning algorithms AI solutions data privacy security\""}

Tool Output

[{'title': '2024: The State of Generative AI in the Enterprise', 'url': 'https://menlovc.com/2024-the-state-of-generative-ai-in-the-enterprise/', 'content': 'Last year, our 2023 State of Generative AI in the Enterprise Report captured a period of early AI experimentation. Our 2024 report, expanded to include insights from 600 U.S. enterprise leaders, reveals the trends emerging as organizations shift from pilots to production, making AI an enterprise imperative.\n\n## Generative AI Spending Signals Growing Enterprise Commitment [...] Today, 60% of enterprise generative AI investments come from innovation budgets, reflecting the early stages of generative AI adoption. However, with 40% of generative AI spending sourced from more permanent budgets-58% of which is redirected from existing allocations-businesses are demonstrating a growing commitment to AI transformation.\n\nGraph showing
```

```
Tool Input

{"question": "\latest trends in Generative AI for enterprises 2023 machine learning algorithms AI solutions data privacy security\""}

Tool Output

[{'title': '2024: The State of Generative AI in the Enterprise', 'url': 'https://menlovc.com/2024-the-state-of-generative-ai-in-the-enterprise/', 'content': 'Last year, our 2023 State of Generative AI in the Enterprise Report captured a period of early AI experimentation. Our 2024 report, expanded to include insights from 600 U.S. enterprise leaders, reveals the trends emerging as organizations shift from pilots to production, making AI an enterprise imperative.\n\n## Generative AI Spending Signals Growing Enterprise Commitment [...] Today, 60% of enterprise generative AI investments come from innovation budgets, reflecting the early stages of generative AI adoption. However, with 40% of generative AI spending sourced from more permanent budgets-58% of which is redirected from existing allocations-businesses are demonstrating a growing commitment to AI transformation.\n\nGraph showing enterprise generative AI spend by category in 2023 vs. 2024 [...] ## Our Predictions\n\n2024 has been a year of transition and evolution, as the wave of hype we documented in 2023 gave way to real-world implementation. Drawing on the data we've shared today and the trends we've observed as investors, these are our three predictions for what lies ahead:\n\n## This Is Just the Beginning', 'score': 0.735737}, {'title': 'Managing Data Security and Privacy Risks in Enterprise AI', 'url': 'https://frostbrowntodd.com/managing-data-security-and-privacy-risks-in-enterprise-ai/', 'content': 'Privacy and data security in the context of AI are interdependent disciplines that often require simultaneous consideration and action. To begin with, advanced enterprise AI tools are trained on prodigious amounts of data processed using algorithms that should be-but are not always-designed to comply with privacy and security laws and regulations. Those laws, primarily at the state level, are still evolving and can vary considerably from one jurisdiction to the next. Another concern is whether [...] Ar...

Crew: crew
Task: e29f335a-5d0f-4517-b9d7-381735cb89c3
Status: Executing Task...
└─ Used Search tool (1)

Agent Final Answer

Agent: Researcher

Final Answer:
The latest trends in Generative AI (GenAI) for enterprises highlight significant advancements and ongoing challenges:

1. **Generative AI Adoption**:
- A report reveals that while many enterprises are experimenting with generative AI, a transition from pilot tests to production use is emerging as a necessity. Investments in generative AI have increased, with 60% from innovation budgets and a commitment showcased by redirecting funds from existing budgets. Companies are fostering a dedicated approach to AI transformation beyond initial experiments (Source: [Menlo Ventures](https://menlovc.com/2024-the-state-of-generative-ai-in-the-enterprise/)).
```

```
Crew: crew
└─ Task: e29f335a-5d0f-4517-b9d7-381735cb89c3
   Status: Executing Task...
   └─ Used Search tool (1)

Agent Final Answer

Agent: Researcher

Final Answer:
The latest trends in Generative AI (GenAI) for enterprises highlight significant advancements and ongoing challenges:

1. Generative AI Adoption:
   - A report reveals that while many enterprises are experimenting with generative AI, a transition from pilot tests to production use is emerging as a necessity. Investments in generative AI have increased, with 68% from innovation budgets and a commitment showcased by redirecting funds from existing budgets. Companies are fostering a dedicated approach to AI transformation beyond initial experiments (Source: [Menlo Ventures](https://menlovc.com/2024-the-state-of-generative-ai-in-the-enterprise/)).

2. Focus on Data Privacy and Security:
   - Privacy and data security are paramount as AI tools consume vast amounts of data. However, these tools must comply with evolving privacy laws. There's growing concern over the risks associated with sensitive data being targeted by cybercriminals, emphasizing the need for strict security measures and compliance in AI governance (Source: [Frost Brown Todd](https://frostbrowntodd.com/managing-data-security-and-privacy-risks-in-enterprise-ai/)).

3. Machine Learning Algorithms:
   - Businesses are leveraging machine learning algorithms to automate tasks and predict trends. Current trends suggest a dynamic evolution in how these algorithms are applied for strategic decision-making, shaping innovations in various sectors (Source: [Appinventiv](https://appinventiv.com/blog/machine-learning-algorithms-for-business-operations/)).

4. Scaling Challenges:
   - As organizations invest more in generative AI, they are also facing challenges in scaling their projects effectively. Most businesses are proceeding with caution, opting to conduct a limited number of experiments, with realistic expectations on the outcomes and timeline for scaling initiatives (Source: [Deloitte](https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/content/state-of-generative-ai-in-enterprise.html)).

5. Future Outlook:
   - The focus for 2024 has shifted towards tangible results and pragmatic approaches, with organizations balancing excitement over AI capabilities with a cautious and measured strategy for implementation and ROI (Source: [Deloitte](https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/content/state-of-generative-ai-in-enterprise.html)).

Overall, as enterprises continue to adopt generative AI, the dual focus remains on enhancing machine learning capabilities while ensuring robust data privacy and security compliance.
```

```
Crew: crew
└─ Task: e29f335a-5d0f-4517-b9d7-381735cb89c3
   Assigned to: Researcher
   Status: Completed
   └─ Used Search tool (1)

Task Completion

Task Completed
Name: e29f335a-5d0f-4517-b9d7-381735cb89c3
Agent: Researcher
Tool Args:

Crew Completion

Crew Execution Completed
Name: crew
ID: bb1b18d2-47a9-48cb-976d-1246a0e4bcc6
Tool Args:
Final Output: The latest trends in Generative AI (GenAI) for enterprises highlight significant advancements and ongoing challenges:

1. Generative AI Adoption:
   - A report reveals that while many enterprises are experimenting with generative AI, a transition from pilot tests to production use is emerging as a necessity. Investments in generative AI have increased, with 68% from innovation budgets and a commitment showcased by redirecting funds from existing budgets. Companies are fostering a dedicated approach to AI transformation beyond initial experiments (Source: [Menlo Ventures](https://menlovc.com/2024-the-state-of-generative-ai-in-the-enterprise/)).

2. Focus on Data Privacy and Security:
   - Privacy and data security are paramount as AI tools consume vast amounts of data. However, these tools must comply with evolving privacy laws. There's growing concern over the risks associated with sensitive data being targeted by cybercriminals, emphasizing the need for strict security measures and compliance in AI governance (Source: [Frost Brown Todd](https://frostbrowntodd.com/managing-data-security-and-privacy-risks-in-enterprise-ai/)).

3. Machine Learning Algorithms:
   - Businesses are leveraging machine learning algorithms to automate tasks and predict trends. Current trends suggest a dynamic evolution in how these algorithms are applied for strategic decision-making, shaping innovations in various sectors (Source: [Appinventiv](https://appinventiv.com/blog/machine-learning-algorithms-for-business-operations/)).
```

```
Name: crew
ID: bb1b18d2-47a9-48cb-976d-1246a0e4bcc6
Tool Args:
Final Output: The latest trends in Generative AI (GenAI) for enterprises highlight significant advancements and ongoing challenges:

1. Generative AI Adoption:
   - A report reveals that while many enterprises are experimenting with generative AI, a transition from pilot tests to production use is emerging as a necessity. Investments in generative AI have increased, with 68% from innovation budgets and a commitment showcased by redirecting funds from existing budgets. Companies are fostering a dedicated approach to AI transformation beyond initial experiments (Source: [Menlo Ventures](https://menlovc.com/2024-the-state-of-generative-ai-in-the-enterprise/)).

2. Focus on Data Privacy and Security:
   - Privacy and data security are paramount as AI tools consume vast amounts of data. However, these tools must comply with evolving privacy laws. There's growing concern over the risks associated with sensitive data being targeted by cybercriminals, emphasizing the need for strict security measures and compliance in AI governance (Source: [Frost Brown Todd](https://frostbrowntodd.com/managing-data-security-and-privacy-risks-in-enterprise-ai/)).

3. Machine Learning Algorithms:
   - Businesses are leveraging machine learning algorithms to automate tasks and predict trends. Current trends suggest a dynamic evolution in how these algorithms are applied for strategic decision-making, shaping innovations in various sectors (Source: [Appinventiv](https://appinventiv.com/blog/machine-learning-algorithms-for-business-operations/)).

4. Scaling Challenges:
   - As organizations invest more in generative AI, they are also facing challenges in scaling their projects effectively. Most businesses are proceeding with caution, opting to conduct a limited number of experiments, with realistic expectations on the outcomes and timeline for scaling initiatives (Source: [Deloitte](https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/content/state-of-generative-ai-in-enterprise.html)).

5. Future Outlook:
   - The focus for 2024 has shifted towards tangible results and pragmatic approaches, with organizations balancing excitement over AI capabilities with a cautious and measured strategy for implementation and ROI (Source: [Deloitte](https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/content/state-of-generative-ai-in-enterprise.html)).

Overall, as enterprises continue to adopt generative AI, the dual focus remains on enhancing machine learning capabilities while ensuring robust data privacy and security compliance.

Final Output:
However, organizations are facing challenges in scaling their operations effectively. Looking ahead, businesses are anticipated to adopt a more pragmatic approach to AI integration, focusing on tangible results and a calculated strategy for implementation and return on investment.
```

By completing these steps, you have integrated multiple agents with LangGraph using the Google A2A protocol to build a modular, seamless workflow for enterprise-ready AI orchestration.